



# Hybrid-NL2SVA

A RAG+Self-Correction Pipeline for NL2SVA, Cross-Model Evaluation,  
and an End-to-End Spec2SVA Extension via AssertionForge

Hybrid-NL2SVA Project

New York University

September 3, 2026

# Outline

---

Motivation

Pipeline Architecture

Key Mechanisms

Evaluation Setup

Results: RAG+SOR Pipeline

Motivation

AssertionForge: Spec2NL

Integration Architecture

UART Pilot

Results

Next Steps

Part I

# The Hybrid-NL2SVA Pipeline

# Why NL2SVA is hard

---

**The task.** Given

- an RTL design (or testbench), and
- a natural-language verification property,

produce a SystemVerilog Assertion (SVA) that is *functionally equivalent* to what a verification engineer intended.

**Why general-purpose LLMs struggle.**

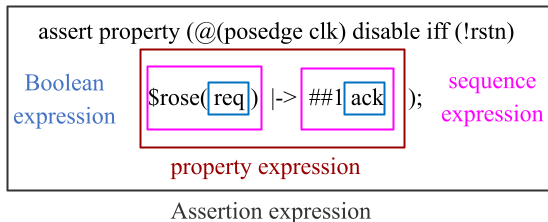
- SVA has subtle, easy-to-confuse temporal operators ( $| \rightarrow$  vs.  $| \Rightarrow$ ,  $\#\#N$  vs.  $\#\#[M:N]$ ,  $s\_eventually$  vs. plain *eventually*)
- Operator precedence traps (and/or binding tighter than  $| \rightarrow$ )
- Little SVA-specific data in general pretraining corpora

**Our approach:** don't just prompt a general LLM once – retrieve grounded operator knowledge, generate, then *mechanically verify and self-correct* against a real formal tool (JasperGold) before ever trusting the answer.

# The anatomy of an SVA

Four nested layers, each with its own operators – the source of most confusion

---

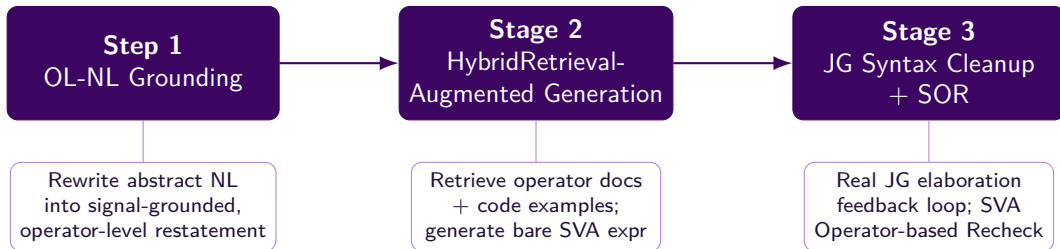


A Boolean expression (`req`) is wrapped by a sequence expression (`$rose(req) |-> ##1 ack`), wrapped by a property expression, wrapped by the assertion statement itself – getting any one layer's operator wrong (e.g. a sequence operator used where a property operator belongs) silently changes the property's meaning rather than raising a syntax error.

# Recap: the original three-stage pipeline

The baseline this talk's improvements build on top of

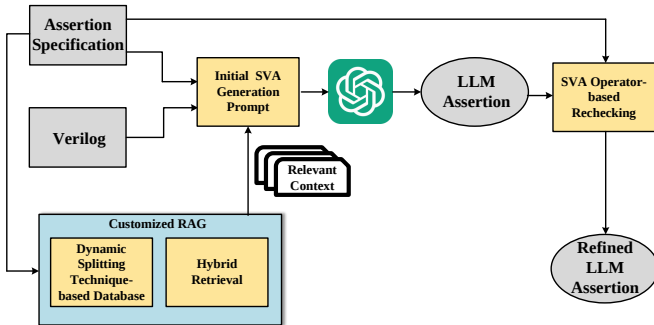
---



The model is *never* trusted to write the `clock/disable iff/label` itself (`EXPRESSION_ONLY_INSTRUCTION`) — these are mechanically attached (`wrap_property_expression`) once the bare boolean/temporal expression is finalized.

# As published: the overall pipeline

The customized RAG framework, TCAD paper Fig. 2



*Customized RAG* bundles two techniques: **dynamic splitting** (builds a code-centric retrieval database from SVA textbooks, preserving each code snippet with its surrounding explanation) and **Hybrid Retrieval** (next slide). Retrieved context enriches the Initial SVA Generation Prompt; the output is then passed through SVA Operator-based Rechecking (Stage 3) for a refined final assertion.

# Stage 1 in detail: OL-NL grounding

`--ol-nl-grounding` – only needed when the input isn't already signal-grounded

---

“OL NL” = **operator-level natural language**: a restatement naming *only* signals that actually appear in the testbench (never invented/ paraphrased names), stating the property's structure as precisely as possible – condition first, then outcome. One best-effort LLM call, no golden SVA shown.

## Worked example:

Description: that the pending flag never gets stuck.

Signals: pending, pending\_d1, clear\_vld, sat\_guard

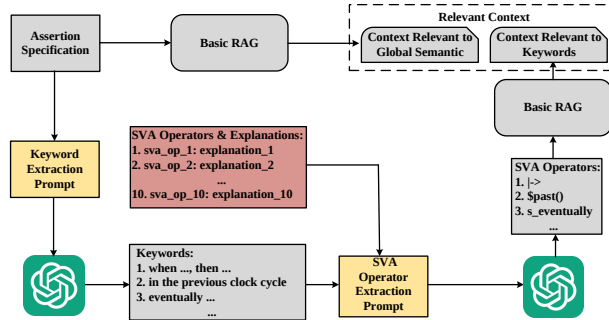
OL NL: It must never be the case that pending\_d1 is asserted and clear\_vld is not asserted and pending is still asserted and sat\_guard is not asserted

**When it runs:** needed for `nl2sva_human_verified`; *redundant* for `nl2sva_machine_verified` (already names every signal) – skipped there. `--ol-nl-conservative`: stay maximally literal, never invent structure the description doesn't state.



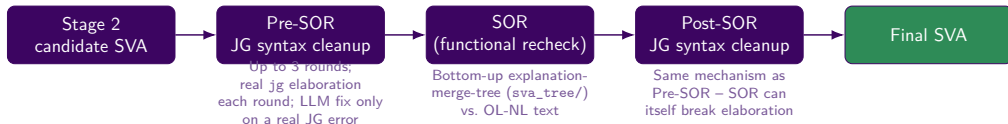
# Stage 2 in detail: HybridRetrieval-augmented generation

Two independent retrieval paths feed one generation call – TCAD paper Fig. 3



**Path 1 (top):** Basic RAG on the full assertion specification – global-semantic similarity against the code database. **Path 2 (bottom):** an LLM decomposes the description into keyword phrases (*Keyword Extraction Prompt*), maps each to the closest of 10 SVA operators (*SVA Operator Extraction Prompt*), then a second Basic RAG pass retrieves chunks specifically discussing those operators. Both paths' **Relevant Context** feeds the single generation call.

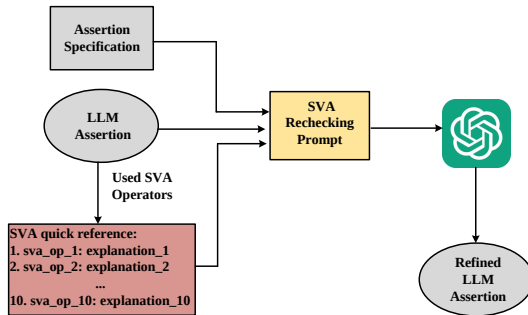
## Stage 3 in detail: syntax cleanup + SOR



**Confirmed live (Human-69):** SOR combined two implications with an invalid `else` inside a property – without the post-SOR pass, this went straight through as the final answer. Running cleanup *twice* (once before, once after SOR) closes that gap.

# As published: SVA operator-based rechecking

TCAD paper Fig. 5 – the originally published SOR mechanism

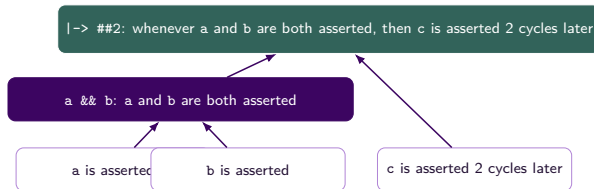


Extract the operators the LLM assertion actually used, look each one up in the SVA quick-reference table, and hand the assertion, the specification, and those explanations to a *SVA Rechecking Prompt*. **Since published:** the explanation-merge-tree (next slide) replaced this flat operator lookup with a full bottom-up recomposition of the assertion's meaning, catching mismatches the original per-operator check could miss.

# Inside SOR: the explanation-merge-tree

Parse the candidate's own operator structure bottom-up, then compare

Candidate SVA: `a && b |-> ##2 c`



**Compare** the root's composed meaning against the Stage 1 OL-NL description. Match → confirm verbatim. Mismatch → the model revises – `--sor-conservative` tells it to change *only* the flagged node (e.g. just `##2`), not rewrite the whole property.

# The bug this talk's improvement targets

A confirmed, systematic timing error in  $\mid\Rightarrow + \#\#N$  composition

---

Composing  $\mid\Rightarrow$  (which itself already advances one clock cycle) with an explicit  $\#\#N$  delay silently lands on  $N+1$  total cycles instead of  $N$  – both at **generation time** and inside **SOR's own revision step**, so the error can survive the self-correction pass meant to catch it.

**Fix (`--only-overlap-implication`):** sidestep it at the root – always use  $\mid\rightarrow$ , express any delay explicitly via  $\#\#N$  using the *total* cycle count, never combine an implicit-advance operator with an explicit one.

`--sor-template-timing`: SOR's explanation-merge-tree can also use fixed, deterministic, LLM-free templates (`sva_temporal_operators.json`'s per-operator fields) instead of an LLM composing each node's meaning – removes another source of the same timing slip.

# The improvement: SOR-timing fixes vs. the original pipeline

Same benchmark rows, same pipeline, only the timing-fix flags differ

| Dataset                           | Configuration             | FM-strict              | FM-relaxed             |
|-----------------------------------|---------------------------|------------------------|------------------------|
| nl2sva_human_<br>verified (73)    | Original pipeline         | 71.2–74.0%             | 83.6–87.7%             |
|                                   | + <b>SOR-timing fixes</b> | <b>74.0%</b>           | <b>87.7%</b>           |
| nl2sva_machine_<br>verified (283) | Original pipeline         | 82.0% (232/283)        | 90.5% (256/283)        |
|                                   | + <b>SOR-timing fixes</b> | <b>85.2% (241/283)</b> | <b>95.4% (270/283)</b> |

sva\_graph.py also had a real, unrelated parsing bug fixed along the way: range/unbounded delays (##[M:N], ##[\*], ##[+]) were mislabeled "##None" internally, discarding their real bounds.

## Benchmark & metrics

---

**FVEval-Verified** (wyt2000/FVEval-Verified): a human-expert-corrected version of FVEval's `nl2sva_human` and `nl2sva_machine` benchmarks.

- **nl2sva\_human\_verified** (73 rows) – expert-written ground-truth SVAs and NL descriptions
- **nl2sva\_machine\_verified** (283 rows) – rule-generated SVAs, LLM-written NL descriptions; bare dummy testbenches, no reset signal

**Metrics** (all via a real JasperGold `prop_eq_checker`):

- **SC** (Syntax Correctness) – does the candidate elaborate against the real testbench on its own?
- **FM-strict** – formally proven full equivalence to the golden SVA
- **FM-relaxed** – FM-strict plus one-directional `implies` matches

# Cross-model comparison — RAG+SOR pipeline

Each row uses that dataset's best-known flag configuration

| Model                        | Dataset | SC                   | FM-strict              | FM-relaxed             |
|------------------------------|---------|----------------------|------------------------|------------------------|
| gpt-4o                       | human   | 98.6% (72/73)        | 74.0% (54/73)          | 87.7% (64/73)          |
| gpt-4o                       | machine | 99.6% (282/283)      | 84.8% (240/283)        | 94.7% (268/283)        |
| DeepSeek-V4-Flash (thinking) | human   | 100.0% (73/73)       | <b>86.3%</b> (63/73)   | <b>94.5%</b> (69/73)   |
| DeepSeek-V4-Flash (thinking) | machine | 100.0% (283/283)     | <b>92.9%</b> (263/283) | <b>98.6%</b> (279/283) |
| gpt-5                        | human   | 100.0% (73/73)       | 82.2% (60/73)          | 91.8% (67/73)          |
| gpt-5                        | machine | interrupted (45/283) | –                      | –                      |
| deepseek-chat (thinking off) | human   | 100.0% (73/73)       | 74.0% (54/73)          | 80.8% (59/73)          |
| deepseek-chat (thinking off) | machine | 98.9% (280/283)      | 85.5% (242/283)        | 94.3% (267/283)        |
| qwen3-8b (Aliyun)            | human   | 91.8% (67/73)        | 35.6% (26/73)          | 53.4% (39/73)          |
| qwen3-8b (Aliyun)            | machine | 88.7% (251/283)      | 60.8% (172/283)        | 73.9% (209/283)        |
| qwen3-14b (Aliyun)           | human   | 91.8% (67/73)        | 65.8% (48/73)          | 79.5% (58/73)          |
| qwen3-14b (Aliyun)           | machine | 95.1% (269/283)      | 81.6% (231/283)        | 90.8% (257/283)        |

DeepSeek-V4-Flash (thinking) was originally run and labeled “DeepSeek-R1” – DeepSeek has since silently remapped the deepseek-reasoner API alias to route to V4-Flash with thinking mode on; relabeled accordingly.



# Cross-model comparison — raw model output (no pipeline)

Same JasperGold scorer, no RAG / OL-NL grounding / SOR

| Model                                | Dataset | SC               | FM-strict       | FM-relaxed      |
|--------------------------------------|---------|------------------|-----------------|-----------------|
| gpt-4o (0-shot)                      | human   | 94.5% (69/73)    | 63.0% (46/73)   | 76.7% (56/73)   |
| CodeV-SVA-8B                         | human   | 97.3% (71/73)    | 75.3% (55/73)   | 83.6% (61/73)   |
| CodeV-SVA-14B                        | human   | 93.2% (68/73)    | 74.0% (54/73)   | 80.8% (59/73)   |
| CodeV-SVA-8B                         | machine | 96.5% (273/283)  | 83.0% (235/283) | 93.3% (264/283) |
| CodeV-SVA-14B                        | machine | 97.2% (275/283)  | 80.9% (229/283) | 93.3% (264/283) |
| DeepSeek-V4-Flash (thinking, 0-shot) | human   | 98.6% (72/73)    | 76.7% (56/73)   | 93.2% (68/73)   |
| DeepSeek-V4-Flash (thinking, 0-shot) | machine | 100.0% (283/283) | 90.8% (257/283) | 98.2% (278/283) |
| deepseek-chat (thinking off, 0-shot) | human   | 94.5% (69/73)    | 54.8% (40/73)   | 76.7% (56/73)   |
| deepseek-chat (thinking off, 0-shot) | machine | 97.5% (276/283)  | 78.4% (222/283) | 89.4% (253/283) |
| qwen3-8b (Aliyun, base)              | human   | 89.0% (65/73)    | 35.6% (26/73)   | 57.5% (42/73)   |
| qwen3-8b (Aliyun, base)              | machine | 87.6% (248/283)  | 51.2% (145/283) | 66.8% (189/283) |
| qwen3-14b (Aliyun, base)             | human   | 93.2% (68/73)    | 49.3% (36/73)   | 69.9% (51/73)   |
| qwen3-14b (Aliyun, base)             | machine | 91.9% (260/283)  | 65.7% (186/283) | 76.3% (216/283) |

**Takeaway:** the pipeline (RAG + OL-NL grounding + SOR) lifts gpt-4o's FM-strict from 63.0%→74.0% (human) and (with the machine config) from a similarly weak 0-shot baseline to 84.8% – a real, structural gain from retrieval + self-correction, not just a stronger base model.

# Case study: when the pipeline does *not* help

qwen3-8b on nl2sva\_human\_verified

---

## Net effect: a wash.

- FM-strict identical: 26/73 both ways
- FM-relaxed *slightly worse*: 53.4% vs. 57.5%
- 12/73 rows flip fail→pass ...
- ...but a near-equal 12/73 flip pass→fail

## Three regressions found:

- Spurious ##N inserted where none was needed
- A boolean's polarity flipped while “fixing” something else
- A correct plain comparison rewritten into a non-equivalent \$onehot0({...}) form

**Conclusion:** the pipeline's net benefit scales with the base model's own ability to self-correct *without introducing new errors* during revision – not a property that holds unconditionally across models. (nl2sva\_machine\_verified does *not* show this wash for the same model: 60.8% vs. 51.2% FM-strict.)

Part II

# End-to-End Spec2SVA

# From curated benchmarks to real designs

---

## What Part I evaluated:

NL2SVA on FVEval-Verified – properties and RTL are already curated, one property per row, always paired with a golden SVA.

## What real verification looks like:

a *specification document* (PDF, dozens of pages) and a *multi-file RTL design* – no pre-extracted property list, no golden SVA to check against.

**Goal:** close the loop – *Spec2NL* (turn a spec + RTL into a list of natural-language verification properties) feeding directly into *our* NL2SVA pipeline, with JasperGold verifying the result end to end.

# AssertionForge (NVLabs, LAD 2025)

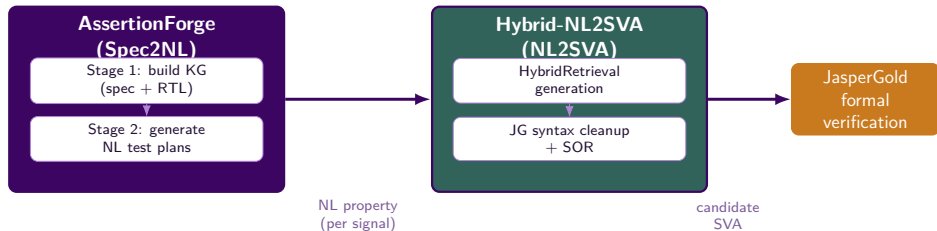
[github.com/NVLabs/AssertionForge](https://github.com/NVLabs/AssertionForge) — [arXiv:2503.19174](https://arxiv.org/abs/2503.19174)

---

- **Stage 1 – Knowledge Graph Construction:** builds a joint KG from the spec (via GraphRAG, domain-customized entity-extraction prompt) and the RTL (pyverilog-based structural analysis), then fuses the two
- **Stage 2 – Test Plan Generation:** for each architectural signal, retrieves KG-guided context (RAG chunks + graph-path analysis), prunes with an LLM judge, and generates natural-language verification *plans*

Stage 1 + Stage 2 together = **Spec2NL**: spec PDF + RTL directory → a list of natural-language properties, each grounded in a specific signal and its role in the design.

# Spec2NL + NL2SVA = end-to-end Spec2SVA



NL2SVA runs with **the same flag configuration as the `nl2sva_machine_verified` evaluation**:  
`--skip-signal-list-note --sor-conservative --only-overlap-implication, no --ol-nl-grounding` (AssertionForge's plans are already signal-grounded) — plus a new `clock_signal` parameter (real RTL rarely names its clock `clk`).

# Case study: UART

AssertLLM's spec + RTL

---

## Design.

- 6 Verilog modules: `uart2bus_top` (real top), `uart_top`, `baud_gen`, `uart_rx`, `uart_tx`, `uart_parser`
- 246 total signals; 18 architectural (interface) signals across the hierarchy
- Real clock port name: `clock` (not `clk`)

## Spec2NL output.

- Knowledge graph: 1,371 nodes, 1,620 edges (spec + RTL fused)
- **323** natural-language verification properties generated (all 18 signals)
- QiMeng-CodeV-SVA's own Table 5 reports **265** SVAs for UART (GPT-4o Spec2NL + GPT-4o NL2SVA) – same order of magnitude

**A real gap found & fixed along the way:** upstream's `valid_signals` example only listed 2 of 18 signals; and `utils_LLM.py/utils_LLM_client.py` ship as empty “add your own client” stubs – both needed fixing before Stage 1/2 ran end to end.

# Sample generated NL properties

---

- **baud\_clk:** “that after a reset condition, baud\_clk will resume toggling without skipping cycles based on the previously set value of baud\_freq”
- **baud\_freq:** “that baud\_freq maintains a constant value when the global\_clock\_freq and baud\_rate inputs remain unchanged over 20 clock cycles”
- **int\_req / int\_gnt:** bus request/grant handshake properties derived from path analysis between uart2bus\_top's internal bus interface signals

Example NL2SVA output (clock\_signal=clock, no disable iff, matching the machine-dataset config):

```
asrt: assert property (@(posedge clock)
    (new_tx_data && !tx_busy) |-> tx_data
);
```



# UART: full NL2SVA + formal verification results

323-property run via Hybrid-NL2SVA (gpt-4o, machine config), scored with JasperGold

| Spec2NL / NL2SVA                  | #SVA | #SynC       | #Proven    |
|-----------------------------------|------|-------------|------------|
| QiMeng-CodeV-SVA: GPT-4o / GPT-4o | 265  | 186 (70.2%) | 54 (20.4%) |
| Ours – all 18 signals             | 323  | 225 (69.7%) | 60 (18.6%) |

**Caveat:** we independently reconstructed the 18-signal `valid_signals` list (the cloned config only had a 2-signal placeholder); we have no evidence the paper's own run used the same signal scope, so this is not a strict apples-to-apples comparison – only evidence our pipeline lands in a plausible, self-consistent range. #Proven checked via a new `check_sva_proven` utility (`prove -all + get_status`, no golden SVA needed).

# A methodology gap: properties about free inputs

AssertionForge's own few-shot prompt (`get_sva_icl_examples`) is entirely PWDATA-shaped

*“that the input data PWDATA has a value between 83 and 165 ... 3 clock cycles after reset deasserted”* – PWDATA is APB's free bus-input signal; every one of AssertionForge's 5 hardcoded style examples follows this same shape.

| UART signal subset                                                                                                        | n          | #Proven      |
|---------------------------------------------------------------------------------------------------------------------------|------------|--------------|
| Free inputs / out-of-scope ( <code>ser_in</code> , <code>int_rd_data</code> , <code>int_gnt</code> , <code>ce_16</code> ) | 67         | 11.9%        |
| <b>Design-controlled only</b> (outputs + internal wires)                                                                  | <b>256</b> | <b>20.3%</b> |

A primary *input* port is free/unconstrained under JasperGold with no environment assumes – a property claiming “this input settles into range X” is essentially unprovable by construction, regardless of NL2SVA quality. Excluding these 4 signals lifts #Proven from 18.6%→**20.3%**, matching the GPT-4o/GPT-4o reference almost exactly. (`ce_16`: declared inside `uart_top`, one level below our assertion scope `uart2bus_top` – not even a valid identifier there.)

# A second gap: signal hallucination, not just free inputs

Of 98 syntax-fail rows, “X’ is not declared” errors name 68 distinct identifiers

| Category (checked against the 6 real RTL files)                                                                                      | occurrences |
|--------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Real RTL identifiers used at the <i>wrong scope</i> (e.g. <code>ce_16×34</code> , <code>bit_count×26</code> , <code>ce_1×22</code> ) | 21 distinct |
| Pure hallucinations, not found anywhere in the RTL ( <code>local_global_clock_freq</code> , <code>logic_counter</code> , ...)        | 47 distinct |

**But most of this traces back further, to the NL plans themselves:** of 338 undeclared-identifier occurrences, **236 (69.8%)** have their “core” name *literally present in the corresponding NL plan’s own text* – e.g. the `baud_freq` plan on the earlier slide reads “... *using the formula* `16*baud_rate / gcd(global_clock_freq, 16*baud_rate)`”, and NL2SVA just faithfully transcribed `baud_rate/global_clock_freq` as identifiers – except neither is a real RTL signal. This is a Spec2NL grounding gap, not (only) an NL2SVA one.

# Root cause: real RTL documentation, not noise

baud\_gen.v's own header comment, verbatim

---

```
// the two registers should be calculated as follows:  
//   baud_freq  = 16*baud_rate / gcd(global_clock_freq, 16*baud_rate)  
//   baud_limit = (global_clock_freq / gcd(...)) - baud_freq
```

baud\_freq is an *input* to baud\_gen – baud\_rate/global\_clock\_freq never appear as actual Verilog identifiers anywhere in the 6 files, only in this comment. True, directly-relevant content, not retrieval noise – the model has good reason to want to cite it.

**Prompt-only fixes made this WORSE, not better**, across several revisions – consistent with Part I's negative-priming finding: naming the exact bad pattern put that pattern in front of the model right before generation. Invalid-signal leakage rose 14%→22%→27%→30% across successive attempts to ban it in words.

# Fix: two-step generation + a mechanical filter

Reworking AssertionForge's Stage 2 (`gen_plan.py`) directly

---

- **Step 1 (idea):** free-form property ideas, no precision rules at all – just “what’s worth testing”
- **Step 2 (OL-NL grounding):** rewrites Step 1’s ideas into precise, signal-grounded form – valid-signal whitelist, operator-level/sequential-vs-combinational discipline, vague-qualifier / trailing-rationale / gcd bans, the *same* SVA Operator Context table Hybrid-NL2SVA’s own Stage 1 uses – literally the same grounding task Stage 1 performs, on AssertionForge’s side instead
- **Mechanical filter (post-hoc, not a prompt):** drops any grounded plan still referencing a `snake_case`, signal-shaped token not in `valid_signals` – carries none of the negative-priming risk above, since it never goes back into a prompt

**Lesson:** banning a specific bad pattern *in words* tends to backfire; pushing the same constraint *out of the prompt* into a mechanical check is what actually held.

# Validated: same 44 properties, ungrounded vs. grounded

`baud_clk/baud_freq` pilot – identical underlying ideas, index-matched

| Same 44 properties, different generation               | n         | #SynC             | #Proven           |
|--------------------------------------------------------|-----------|-------------------|-------------------|
| Raw ideas + gpt-4o 0-shot baseline (no pipeline)       | 44        | 30 (68.2%)        | 15 (34.1%)        |
| Raw ideas + CodeV-SVA-8B (no pipeline)                 | 44        | 18 (40.9%)        | 12 (27.3%)        |
| Raw ideas + CodeV-SVA-14B (no pipeline)                | 44        | 21 (47.7%)        | 13 (29.5%)        |
| <b>Grounded (Step 2 + filter) + full Hybrid-NL2SVA</b> | <b>44</b> | <b>40 (90.9%)</b> | <b>24 (54.5%)</b> |

Same 44 properties both ways (index-matched) – grounding + the full pipeline is **+22.7pp #SynC**, **+20.4pp #Proven** over the best ungrounded baseline; a specialized NL2SVA model alone (CodeV-SVA) doesn't compensate for hallucinated-signal input the way grounding does. *Scope: `baud_clk/baud_freq` only (2/18 signals) – pilot before the remaining 16 and a full regeneration.*

## Next steps

---

- Run the two-step generation + mechanical filter on the remaining 16 signals and fully regenerate `nl_plans_uart.txt`
- Extend the pilot to the remaining 4 designs from QiMeng-CodeV-SVA's Table 5 (APB, ETHMAC, OPENMSP430, SOCKIT)
- Swap AssertionForge's own NL2SVA backend for a head-to-head comparison at the *same* Spec2NL properties
- Try DeepSeek-V4-Flash / qwen3 as AssertionForge's own Spec2NL backend, not just gpt-4o
- Look into why #SynC (69.7%) trails #SVA – categorize the elaboration failures the same way Part I's row-by-row diffing did

**Thank you**